

HOMework 4

Gaurav Batra
9087090024

Instructions: Use this L^AT_EX file as a template to develop your homework. Submit your homework on time as a single PDF file to **Gradescope**. You can choose any programming language (i.e., Python, R, or MATLAB). Please check Piazza for updates about the homework. *Late submissions and hand-written (or non-typed) homework will not be accepted and will receive 0 credit.*

1 Best Prediction Under 0-1 Loss (10 pts)

Suppose the world generates a single observation $x \sim \text{multinomial}(\theta)$, where the parameter vector $\theta = (\theta_1, \dots, \theta_k)$ with $\theta_i \geq 0$ and $\sum_{i=1}^k \theta_i = 1$. Note $x \in \{1, \dots, k\}$. You know θ and want to predict x . Call your prediction $\hat{x}$. What is your expected 0-1 loss:

$$\mathbb{E}[\mathbb{1}_{\hat{x} \neq x}]$$

using the following two prediction strategies respectively? Prove your answer.

Strategy 1: $\hat{x} \in \arg \max_x \theta_x$, the outcome with the highest probability.

Strategy 2: You mimic the world by generating a prediction $\hat{x} \sim \text{multinomial}(\theta)$. (Hint: your randomness and the world's randomness are independent)

Strategy 1: $\hat{x} \in \arg \max_x \theta_x$

In this strategy, we predict the outcome with the highest probability, i.e.,

$$\hat{x} = \arg \max_x \theta_x.$$

Let $x^* = \arg \max_x \theta_x$ denote this outcome. Then we predict $\hat{x} = x^*$ deterministically.

The probability that our prediction $\hat{x}$ is incorrect (i.e., $\hat{x} \neq x$) is simply the probability that x is not equal to x^* , which occurs with probability $1 - \theta_{x^*}$. Hence, the expected 0-1 loss is:

$$\mathbb{E}[\mathbb{1}_{\hat{x} \neq x}] = 1 - \theta_{x^*}.$$

Thus, for Strategy 1, the expected 0-1 loss is:

$$\mathbb{E}[\mathbb{1}_{\hat{x} \neq x}] = 1 - \max_x \theta_x.$$

Strategy 2: $\hat{x} \sim \text{multinomial}(\theta)$

In this strategy, we generate $\hat{x}$ independently according to the same multinomial distribution with parameter θ . Therefore, both x and $\hat{x}$ are drawn independently from $\text{multinomial}(\theta)$.

The probability that $\hat{x} \neq x$ is the probability that two independent draws from the multinomial distribution $\text{multinomial}(\theta)$ do not match. For each possible outcome i , the probability that both $x = i$ and $\hat{x} = i$ is θ_i^2 (since x and $\hat{x}$ are independent).

Thus, the probability that $\hat{x} = x$ is the sum of these probabilities over all possible outcomes:

$$\Pr(\hat{x} = x) = \sum_{i=1}^k \theta_i^2.$$

Therefore, the probability that $\hat{x} \neq x$ is:

$$\Pr(\hat{x} \neq x) = 1 - \sum_{i=1}^k \theta_i^2.$$

So, for Strategy 2, the expected 0-1 loss is:

$$\mathbb{E}[\mathbb{I}_{\hat{x} \neq x}] = 1 - \sum_{i=1}^k \theta_i^2.$$

2 Best Prediction Under Different Misclassification Losses (6 pts)

Like in the previous question, the world generates a single observation $x \sim \text{multinomial}(\theta)$. Let $c_{ij} \geq 0$ denote the loss you incur, if $x = i$ but you predict $\hat{x} = j$, for $i, j \in \{1, \dots, k\}$. $c_{ii} = 0$ for all i . This is a way to generalize different costs on false positives vs false negatives from binary classification to multi-class classification. You want to minimize your expected loss:

$$\mathbb{E}[c_{x\hat{x}}]$$

Derive your optimal prediction $\hat{x}$.

Let $\Pr(x = i) = \theta_i$, which is the probability that the true class is i . The expected loss if we predict $\hat{x} = j$ is the sum of the misclassification losses weighted by the probabilities of each true class outcome:

$$\mathbb{E}[c_{x\hat{x}} \mid \hat{x} = j] = \sum_{i=1}^k c_{ij} \Pr(x = i) = \sum_{i=1}^k c_{ij} \theta_i.$$

This represents the expected loss incurred by predicting class j , considering the probability of each possible true class and the associated misclassification cost.

To minimize the expected loss, we should select $\hat{x}$ such that it minimizes $\mathbb{E}[c_{x\hat{x}} \mid \hat{x} = j]$ across all possible values of j . Therefore, our optimal prediction $\hat{x}$ is:

$$\hat{x} = \arg \min_{j \in \{1, \dots, k\}} \sum_{i=1}^k c_{ij} \theta_i.$$

3 Language Identification with Naive Bayes (8 pts each)

Implement a character-based Naive Bayes classifier that classifies a document as English, Spanish, or Japanese - all written with the 26 lower case characters and space.

The dataset is languageID.tgz, unpack it. This dataset consists of 60 documents in English, Spanish and Japanese. The correct class label is the first character of the filename: $y \in \{e, j, s\}$. (Note: here each file is a document in corresponding language, and it is regarded as one data.)

We will be using a character-based multinomial Naïve Bayes model. You need to view each document as a bag of characters, including space. We have made sure that there are only 27 different types of printable characters (a to z, and space) – there may be additional control characters such as new-line, please ignore those. Your vocabulary will be these 27 character types. (Note: not word types!)

1. Use files 0.txt to 9.txt in each language as the training data. Estimate the prior probabilities $\hat{p}(y = e)$, $\hat{p}(y = j)$, $\hat{p}(y = s)$ using additive smoothing with parameter $\frac{1}{2}$. Give the formula for additive smoothing with parameter $\frac{1}{2}$ in this case. Print and include in final report the prior probabilities. (Hint: Store all probabilities here and below in $\log()$ internally to avoid underflow. This also means you need to do arithmetic in log-space. But answer questions with probability, not log probability.)

We estimate the prior probabilities for each language class $y \in \{e, j, s\}$ (representing English, Japanese, and Spanish, respectively) using additive smoothing with parameter $\alpha = \frac{1}{2}$. The prior probability of class y , denoted by $\hat{p}(y)$, can be estimated using the following formula:

$$\hat{p}(y = c) = \frac{\text{count of documents with label } c + \alpha}{\text{total number of documents} + \alpha \times \text{number of classes}}$$

where:

- $c \in \{e, j, s\}$ represents each of the classes (English, Japanese, Spanish),
- $\alpha = \frac{1}{2}$ is the smoothing parameter,
- count of documents with label c is the count of training documents in class c ,
- total number of documents = 30 is the total number of training documents across all classes,
- number of classes = 3.

Since there are 10 training documents for each language, we have count of documents with label $e = 10$, count of documents with label $j = 10$, and count of documents with label $s = 10$.

Substituting these values into our formula, we compute each prior probability as follows:

1. For English ($y = e$):

$$\hat{p}(y = e) = \frac{10 + \frac{1}{2}}{30 + \frac{1}{2} \times 3} = \frac{10.5}{31.5} = 0.3333$$

2. For Japanese ($y = j$):

$$\hat{p}(y = j) = \frac{10 + \frac{1}{2}}{30 + \frac{1}{2} \times 3} = \frac{10.5}{31.5} = 0.3333$$

3. For Spanish ($y = s$):

$$\hat{p}(y = s) = \frac{10 + \frac{1}{2}}{30 + \frac{1}{2} \times 3} = \frac{10.5}{31.5} = 0.3333$$

Thus, the prior probabilities for each class, in non-log form, are:

$$\hat{p}(y = e) = 0.3333, \quad \hat{p}(y = j) = 0.3333, \quad \hat{p}(y = s) = 0.3333$$

2. Using the same training data, estimate the class conditional probability (multinomial parameter) for English

$$\theta_{i,e} := \hat{p}(c_i | y = e)$$

where c_i is the i -th character. That is, $c_1 = a, \dots, c_{26} = z, c_{27} = \text{space}$. Again use additive smoothing with parameter $\frac{1}{2}$. Give the formula for additive smoothing with parameter $\frac{1}{2}$ in this case. Print θ_e and include in final report which is a vector with 27 elements.

To estimate the class conditional probability $\theta_{i,e}$, which represents the probability of observing character c_i given that the document is in English ($y = e$), we use a multinomial Naïve Bayes model with additive smoothing. Here, each document is treated as a bag of characters, and we aim to estimate $\theta_{i,e} := \hat{p}(c_i | y = e)$ for each character c_i , where $c_1 = a, \dots, c_{26} = z, c_{27} = \text{space}$.

Using additive smoothing with parameter $\alpha = \frac{1}{2}$, the conditional probability of character c_i given class $y = e$ is calculated as:

$$\theta_{i,e} = \hat{p}(c_i | y = e) = \frac{\text{count}(c_i \text{ in English documents}) + \alpha}{\sum_{j=1}^{27} \text{count}(c_j \text{ in English documents}) + 27 \cdot \alpha}$$

where:

- $\text{count}(c_i \text{ in English documents})$ is the total count of character c_i across all English training documents,
- $\sum_{j=1}^{27} \text{count}(c_j \text{ in English documents})$ is the total count of all 27 characters across all English training documents,
- 27 is the number of unique characters in our vocabulary (26 letters plus space),
- $\alpha = \frac{1}{2}$ is the smoothing parameter.

The vector $\theta_e = [\theta_{1,e}, \theta_{2,e}, \dots, \theta_{27,e}]$ represents the estimated conditional probabilities for each character in English. Using the formula above, we calculate each $\theta_{i,e}$ and store them in θ_e .

$$\theta_e = \begin{bmatrix} 0.06016851 \\ 0.01113497 \\ 0.02151 \\ 0.02197258 \\ 0.10536924 \\ 0.01893276 \\ 0.01747894 \\ 0.04721626 \\ 0.05541054 \\ 0.00142078 \\ 0.00373369 \\ 0.02897737 \\ 0.02051875 \\ 0.05792169 \\ 0.0644639 \\ 0.01675202 \\ 0.0005617 \\ 0.05382455 \\ 0.06618206 \\ 0.08012556 \\ 0.02666446 \\ 0.00928465 \\ 0.01549645 \\ 0.00115645 \\ 0.01384437 \\ 0.00062779 \\ 0.17924996 \end{bmatrix} \quad (1)$$

3. Print θ_j, θ_s and include in final report the class conditional probabilities for Japanese and Spanish.

$$\theta_j = \begin{bmatrix} 1.31765610e-01 \\ 1.08669066e-02 \\ 5.48586603e-03 \\ 1.72263182e-02 \\ 6.02047591e-02 \\ 3.87854223e-03 \\ 1.40116706e-02 \\ 3.17621161e-02 \\ 9.70334393e-02 \\ 2.34110207e-03 \\ 5.74094133e-02 \\ 1.43261470e-03 \\ 3.97987351e-02 \\ 5.67105769e-02 \\ 9.11632132e-02 \\ 8.73545547e-04 \\ 1.04825466e-04 \\ 4.28037318e-02 \\ 4.21747790e-02 \\ 5.69901115e-02 \\ 7.06174220e-02 \\ 2.44592753e-04 \\ 1.97421294e-02 \\ 3.49418219e-05 \\ 1.41514379e-02 \\ 7.72214263e-03 \\ 1.23449457e-01 \end{bmatrix} \quad (2)$$

$$\theta_s = \begin{bmatrix} 1.04560451e-01 \\ 8.23286362e-03 \\ 3.75258241e-02 \\ 3.97459221e-02 \\ 1.13810860e-01 \\ 8.60287996e-03 \\ 7.18448398e-03 \\ 4.53270019e-03 \\ 4.98597021e-02 \\ 6.62945947e-03 \\ 2.77512257e-04 \\ 5.29431717e-02 \\ 2.58086399e-02 \\ 5.41765595e-02 \\ 7.24923684e-02 \\ 2.42669051e-02 \\ 7.67783910e-03 \\ 5.92951189e-02 \\ 6.57704049e-02 \\ 3.56140730e-02 \\ 3.37023219e-02 \\ 5.88942678e-03 \\ 9.25040856e-05 \\ 2.49761031e-03 \\ 7.86284728e-03 \\ 2.68261848e-03 \\ 1.68264932e-01 \end{bmatrix} \quad (3)$$

4. Treat e10.txt as a test document x . Represent x as a bag-of-words count vector (Hint: the vocabulary has size 27). Print the bag-of-words vector x and include in final report.

$$\theta_s = \begin{bmatrix} 164. \\ 32. \\ 53. \\ 57. \\ 311. \\ 55. \\ 51. \\ 140. \\ 140. \\ 3. \\ 6. \\ 85. \\ 64. \\ 139. \\ 182. \\ 53. \\ 3. \\ 141. \\ 186. \\ 225. \\ 65. \\ 31. \\ 47. \\ 4. \\ 38. \\ 2. \\ 498. \end{bmatrix} \quad (4)$$

5. Compute $\hat{p}(x \mid y)$ for $y = e, j, s$ under the multinomial model assumption, respectively. Use the formula

$$\hat{p}(x \mid y) = \prod_{i=1}^d \theta_{i,y}^{x_i}$$

where $x = (x_1, \dots, x_d)$. Show the three values: $\hat{p}(x \mid y = e)$, $\hat{p}(x \mid y = j)$, $\hat{p}(x \mid y = s)$. Hint: you may notice that we omitted the multinomial coefficient. This is ok for classification because it is a constant w.r.t. y .

To compute the probabilities $\hat{p}(x \mid y)$ for $y = e$, $y = j$, and $y = s$ under the multinomial model assumption, we utilize the formula:

$$\hat{p}(x \mid y) = \prod_{i=1}^d \theta_{i,y}^{x_i}$$

where $x = (x_1, x_2, \dots, x_d)$ is the bag-of-words count vector, and $\theta_{i,y}$ represents the class conditional probabilities for character c_i given class y .

Given the previously calculated values of $\theta_{i,e}$, $\theta_{i,j}$, and $\theta_{i,s}$ (from the training data), we can compute $\hat{p}(x \mid y)$ as follows:

- For $y = e$:

$$\hat{p}(x \mid y = e) = \prod_{i=1}^{27} \theta_{i,e}^{x_i}$$

- For $y = j$:

$$\hat{p}(x | y = j) = \prod_{i=1}^{27} \theta_{i,j}^{x_i}$$

- For $y = s$:

$$\hat{p}(x | y = s) = \prod_{i=1}^{27} \theta_{i,s}^{x_i}$$

Therefore,

$$\log \hat{p}(x|y = e) = -7841.8654470606$$

$$\log \hat{p}(x|y = j) = -8771.4330790750$$

$$\log \hat{p}(x|y = s) = -8467.2820440106$$

As probabilities are really small, I have show the log values.

Hence,

$$\hat{p}(x|y = e) \sim 0$$

$$\hat{p}(x|y = j) \sim 0$$

$$\hat{p}(x|y = s) \sim 0$$

6. Use Bayes rule and your estimated prior and likelihood, compute the posterior $\hat{p}(y | x)$. Show the three values: $\hat{p}(y = e | x)$, $\hat{p}(y = j | x)$, $\hat{p}(y = s | x)$. Show the predicted class label of x .

To compute the posterior probabilities $\hat{p}(y | x)$ for each class y , we use Bayes' rule:

$$\hat{p}(y | x) = \frac{\hat{p}(x | y) \cdot \hat{p}(y)}{\hat{p}(x)}$$

Where: - $\hat{p}(y)$ is the prior probability of class y . - $\hat{p}(x | y)$ is the likelihood of the document given class y . - $\hat{p}(x)$ is the evidence, which can be computed as:

$$\hat{p}(x) = \hat{p}(x | y = e) \cdot \hat{p}(y = e) + \hat{p}(x | y = j) \cdot \hat{p}(y = j) + \hat{p}(x | y = s) \cdot \hat{p}(y = s)$$

Let $\hat{p}(x | y = e)$, $\hat{p}(x | y = j)$, and $\hat{p}(x | y = s)$ be the likelihoods computed previously.

Substituting these values into Bayes' rule, we compute the posterior probabilities:

$$\hat{p}(y = e | x) = \frac{\hat{p}(x | y = e) \cdot \hat{p}(y = e)}{\hat{p}(x)}$$

$$\hat{p}(y = j | x) = \frac{\hat{p}(x | y = j) \cdot \hat{p}(y = j)}{\hat{p}(x)}$$

$$\hat{p}(y = s | x) = \frac{\hat{p}(x | y = s) \cdot \hat{p}(y = s)}{\hat{p}(x)}$$

The predicted class label for the document x is the class y that maximizes the posterior probability:

$$\hat{y} = \arg \max_{y \in \{e,j,s\}} \hat{p}(y | x)$$

Therefore,

$$\hat{p}(y = e|x) = 1.0$$

$$\hat{p}(y = j|x) = 0.0$$

$$\hat{p}(y = s|x) = 0.0$$

Hence, the class label is e .

7. Evaluate the performance of your classifier on the test set (files 10.txt to 19.txt in three languages). Present the performance using a confusion matrix. A confusion matrix summarizes the types of errors your classifier makes, as shown in the table below. The columns are the true language a document is in, and the rows are the classified outcome of that document. The cells are the number of test documents in that situation. For example, the cell with row = English and column = Spanish contains the number of test documents that are really Spanish, but misclassified as English by your classifier.

The confusion matrix summarizes the classification outcomes as follows:

Predicted \ True	English (e)	Japanese (j)	Spanish (s)
English (e)	10	0	0
Japanese (j)	0	10	0
Spanish (s)	0	0	10

Where each entry represents the number of documents classified in that manner. For instance, the cell in row "English" and column "Spanish" indicates the count of documents that are actually Spanish but were predicted as English.

The confusion matrix was visualized and can be found in Figure 1.

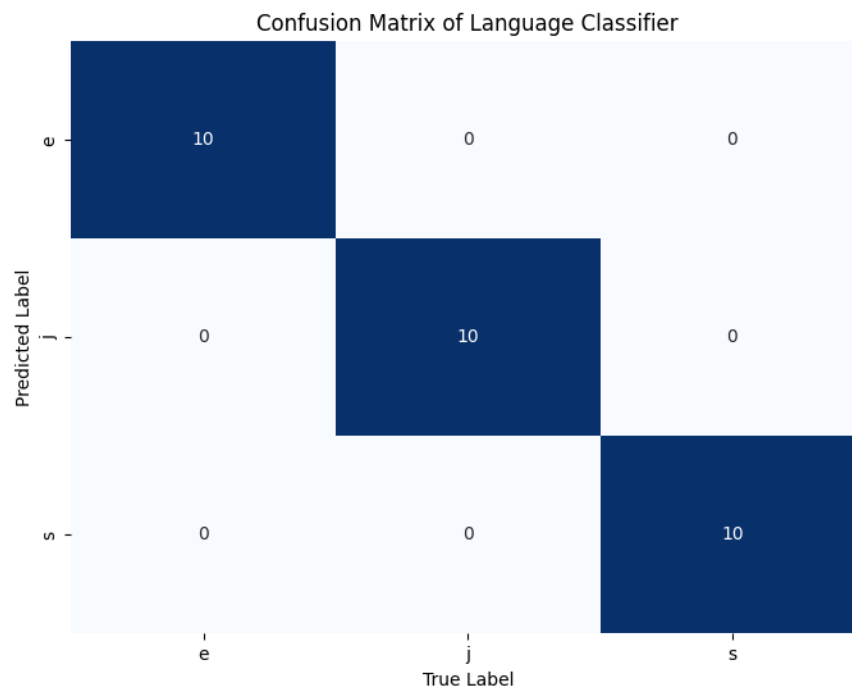


Figure 1: Confusion Matrix of the Naive Bayes Classifier

8. If you take a test document and arbitrarily shuffle the order of its characters so that the words (and spaces) are scrambled beyond human recognition. How does this shuffling affect your Naive Bayes classifier's prediction on this document? Explain the key mathematical step in the Naive Bayes model that justifies your answer.

If we take a test document and arbitrarily shuffle the order of its characters, such that the words and spaces are scrambled beyond human recognition, this shuffling does not affect the predictions made by our Naive Bayes classifier.

The key mathematical step that justifies this conclusion lies in the way the Naive Bayes classifier represents documents. Specifically, it employs the bag-of-words model, which treats documents as collections of characters, disregarding their order. The classifier computes the conditional probability of the document given a class as follows:

$$\hat{p}(x | y) = \prod_{i=1}^d \theta_{i,y}^{x_i}$$

Here, $x = (x_1, x_2, \dots, x_d)$ represents the bag-of-words count vector, where x_i is the count of character c_i in the document and d is the size of the vocabulary (27 characters in our case). The term $\theta_{i,y}$ denotes the class conditional probability of character c_i given class y .

Since the probability $\hat{p}(x | y)$ is computed as the product of the character counts raised to the power of their corresponding probabilities, the order of the characters does not influence the result. This means that regardless of how the characters are arranged, the counts of each character remain the same, and thus the classifier's prediction will remain unchanged. Therefore, the prediction made by the Naive Bayes classifier will be invariant to the character order in the document.

4 Simple Feed-Forward Network (20pts)

In this exercise, you will derive, implement back-propagation for a simple neural network and compare your output with some standard library's output. Consider the following 3-layer neural network.

$$\hat{y} = f(x) = g(W_2 \sigma(W_1 x))$$

Suppose $x \in \mathbb{R}^d$, $W_1 \in \mathbb{R}^{d_1 \times d}$, and $W_2 \in \mathbb{R}^{k \times d_1}$ i.e. $f: \mathbb{R}^d \rightarrow \mathbb{R}^k$. Let $\sigma(z) = [\sigma(z_1), \dots, \sigma(z_n)]$ for any $z \in \mathbb{R}^n$ where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid (logistic) activation function and $g(z_i) = \frac{\exp(z_i)}{\sum_{i=1}^k \exp(z_i)}$ is the softmax function. Suppose the true pair is (x, y) where $y \in \{0, 1\}^k$ with exactly one of the entries equal to 1, and you are working with the cross-entropy loss function given below,

$$L(x, y) = - \sum_{i=1}^k y \log(\hat{y}_i)$$

1. Derive backpropagation updates for the above neural network. (5 pts)

The network computes the following steps:

$$z^{(1)} = W_1 x$$

$$a^{(1)} = \sigma(z^{(1)})$$

$$z^{(2)} = W_2 a^{(1)}$$

$$\hat{y} = g(z^{(2)})$$

The loss function is the cross-entropy loss:

$$L(x, y) = - \sum_{i=1}^k y_i \log(\hat{y}_i)$$

Derivative of Loss with Respect to Output $\hat{y}$

$$\frac{\partial L}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i}$$

Derivative of $\hat{y}$ with Respect to $z^{(2)}$

$$\frac{\partial \hat{y}_i}{\partial z_j^{(2)}} = \begin{cases} \hat{y}_i(1 - \hat{y}_i) & \text{if } i = j \\ -\hat{y}_i\hat{y}_j & \text{if } i \neq j \end{cases}$$

Derivative of L with Respect to $z^{(2)}$

$$\frac{\partial L}{\partial z_j^{(2)}} = \hat{y}_j - y_j$$

Derivative of L with Respect to W_2 and $a^{(1)}$

$$\frac{\partial L}{\partial W_2} = (\hat{y} - y) \cdot (a^{(1)})^T$$

Derivative of L with Respect to $z^{(1)}$

$$\delta^{(1)} = (W_2^T(\hat{y} - y)) \odot \sigma'(z^{(1)})$$

Derivative of L with Respect to W_1

$$\frac{\partial L}{\partial W_1} = \delta^{(1)} \cdot x^T$$

$$W_2 := W_2 - \alpha \frac{\partial L}{\partial W_2}$$

$$W_1 := W_1 - \alpha \frac{\partial L}{\partial W_1}$$

2. Implement it in NumPy or PyTorch using basic linear algebra operations. (e.g. You are not allowed to use auto-grad, built-in optimizer, model, etc. in this step. You can use library functions for data loading, processing, etc.). Evaluate your implementation on MNIST dataset, report test errors and learning curve. (10 pts)

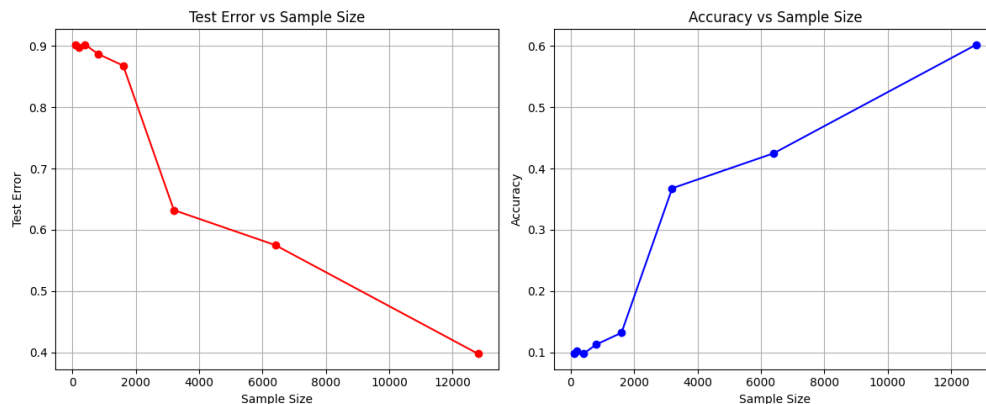


Figure 2: Test Error v/s Sample Size and Accuracy v/s Sample Size

The neural network consists of three layers:

- **Input Layer:**
 - Size: $d = 784$ (28x28 pixel images flattened)
- **Hidden Layer 1:**
 - Size: $d_1 = 300$
 - Activation Function: Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$
- **Hidden Layer 2:**
 - Size: $d_2 = 200$
 - Activation Function: Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$
- **Output Layer:**
 - Size: $k = 10$ (for digits 0-9)
 - Activation Function: Softmax $g(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$

The training was conducted using different sample sizes from the MNIST dataset to evaluate the model's performance:

- Sample Sizes: $\{100, 200, 400, 800, 1600, 3200, 6400, 12800\}$

The weights of the network were initialized using the Xavier initialization method, which is defined as follows:

- **Xavier Initialization:**
 - For each weight W , values are sampled from a uniform distribution in the range $\left[-\frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}, \frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}\right]$, where n_{in} and n_{out} are the number of input and output units, respectively.

3. Implement the same network in PyTorch (or any other framework). You can use all the features of the framework e.g. auto-grad etc. Evaluate it on MNIST dataset, report test errors, and learning curve. (2 pts)

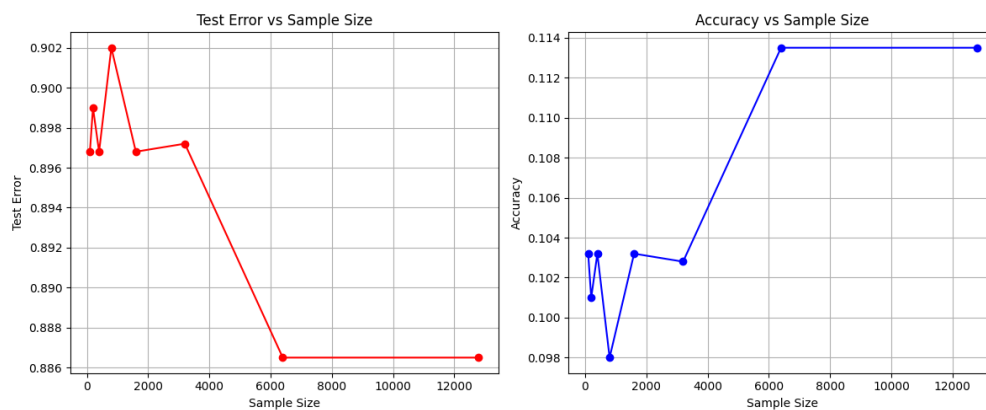


Figure 3: Test Error v/s Sample Size and Accuracy v/s Sample Size

The implemented neural network consists of three fully connected layers:

- **Input Layer:**
 - Size: 784 (for 28×28 pixel MNIST images).

- **Hidden Layer 1:**
 - Size: 300
 - Activation Function: Sigmoid.
- **Hidden Layer 2:**
 - Size: 200
 - Activation Function: Sigmoid.
- **Output Layer:**
 - Size: 10 (for digit classes 0 – 9).
 - Activation Function: Softmax.

The weights of all layers are initialized using **Xavier Initialization** (Glorot initialization), which sets the weights to values drawn from a uniform distribution within the range determined by the number of input and output units in the layer. This approach helps maintain a balanced scale of activations throughout the network and improves convergence during training.

The network is evaluated with varying sample sizes from the MNIST training dataset:

- Sample Sizes: 100, 200, 400, 800, 1600, 3200, 6400, 12800

These sample sizes allow for an analysis of the model's performance and generalization capability as the amount of training data increases.

4. Try different weight initialization a) all weights initialized to 0, and b) initialize the weights randomly between -1 and 1. Report test error and learning curves for both. (You can use either of the implementations) (3 pts)

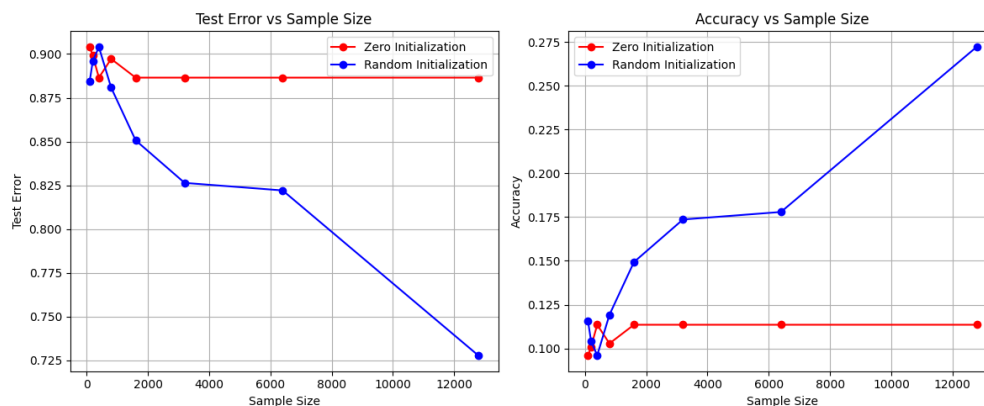


Figure 4: Test Error v/s Sample Size and Accuracy v/s Sample Size for 0 initialized and Random initialized weights between -1 and 1

You should play with different hyperparameters like learning rate, batch size, etc. for your own learning. You only need to report results for any particular setting of hyperparameters. You should mention the values of those along with the results. Use $d_1 = 300$, $d_2 = 200$. For optimization use SGD (Stochastic gradient descent) without momentum, with some batch size say 32, 64, etc. MNIST can be obtained from here (<https://pytorch.org/vision/stable/datasets.html>)